

Protocol & Cryptography – WireGuard

Jason A. Donenfeld

Much more information may be found in the [technical whitepaper](#). For just a quick & dirty overview, read onward here.

Primitives

The following protocols and primitives are used:

- [ChaCha20](#) for symmetric encryption, authenticated with [Poly1305](#), using [RFC7539's AEAD construction](#)
- [Curve25519](#) for ECDH
- [BLAKE2s](#) for hashing and keyed hashing, described in [RFC7693](#)
- [SipHash24](#) for hashtable keys
- [HKDF](#) for key derivation, as described in [RFC5869](#)

Connection-less Protocol

Any secure protocol requires some state to be kept, so there is an initial very simple handshake that establishes symmetric keys to be used for data transfer. This handshake occurs every few minutes, in order to provide rotating keys for perfect forward secrecy. It is done based on time, and not based on the contents of prior packets, because it is designed to deal gracefully with packet loss. There is a clever pulse mechanism to ensure that the latest keys and handshakes are up to date, renegotiating when needed, by automatically detecting when handshakes are out of date. It uses a separate packet queue per host, so that it can minimize packet loss during handshakes while providing steady performance for all clients.

In other words, you bring the device up, and everything else is handled for you automatically. You don't need to worry about asking it to reconnect or disconnect or reinitialize, or anything of that nature.

The following timers are at play:

- A handshake initiation is retried after `REKEY_TIMEOUT + jitter` ms, if a response has not been received, where jitter is some random value between 0 and 333 ms.
- If a packet has been received from a given peer, but we have not sent one back to the given peer in `KEEPALIVE` ms, we send an empty packet.
- If we have sent a packet to a given peer but have not received a packet after from that peer for `KEEPALIVE + REKEY_TIMEOUT` ms, we initiate a new handshake.
- All ephemeral private keys and symmetric session keys are zeroed out after `REJECT_AFTER_TIME * 3` ms if no new keys have been exchanged.
- After sending a packet, if the number of packets sent using that key exceeds `REKEY_AFTER_MESSAGES`, we initiate a new handshake.
- After sending a packet, if the sender was the original initiator of the handshake and if the current session key is `REKEY_AFTER_TIME` ms old, we initiate a new handshake. If the sender was the original responder of the handshake, we do not reinitiate a new handshake after `REKEY_AFTER_TIME` ms like the original initiator does.

- After receiving a packet, if the receiver was the original initiator of the handshake and if the current session key is `REKEY_AFTER_TIME - KEEPALIVE_TIMEOUT - REKEY_TIMEOUT` ms old, we initiate a new handshake.
- Handshakes are only initiated once every `REKEY_TIMEOUT` ms, with this strict rate limiting enforced.
- Packets are dropped if the session counter is greater than `REJECT_AFTER_MESSAGES` or if its key is older than `REJECT_AFTER_TIME` ms.
- After `REKEY_ATTEMPT_TIME` ms of trying to initiate a new handshake, the retries give up and cease, and clear all existing packets queued up to be sent. If a packet is explicitly queued up to be sent, then this timer is reset.

Future work involves adjusting `REKEY_TIMEOUT` to use exponential back-off.

After a handshake is completed, with a message from initiator to responder and then responder back to initiator, the initiator may then send encrypted session packets, but the responder cannot. The responder *must* wait to use the new session until it has received one encrypted session packet from the initiator, in order to provide key confirmation. Thus, until the responder receives that first packet using the newly established session, it must either queue up packets to be sent later, or use the previous session, if one exists and is valid. Therefore, after the initiator receives the response from the responder, if it has no data packets immediately queued up to send, it should send an empty packet, so as to provide this confirmation.

Key Exchange and Data Packets

WireGuard uses the Noise_IK handshake from [Noise](#), building on the work of [CurveCP](#), [NaCL](#), [KEA+](#), [SIGMA](#), [FHMVQ](#), and [HOMVQ](#). All packets are sent over UDP.

The key exchange has these nice properties:

- Avoids key-compromise impersonation
- Avoids replay attacks
- Perfect forward secrecy
- Achieves "AKE security"
- Identity hiding

If an additional layer of symmetric-key crypto is required (for, say, post-quantum resistance), WireGuard also supports an optional pre-shared key that is mixed into the public key cryptography. When pre-shared key mode is *not* in use, the pre-shared key value used below is assumed to be an all-zero string of 32 bytes.

For the following packet descriptions, refer to these functions:

- `DH(private key, public key)`: Curve25519 point multiplication of private key and public key, returning 32 bytes of output
- `DH_GENERATE()`: generate a random Curve25519 private key, returning 32 bytes of output
- `RAND(len)`: return len random bytes of output
- `DH_PUBKEY(private key)`: calculate a Curve25519 public key from private key, returning 32 bytes of output
- `AEAD(key, counter, plain text, auth text)`: ChaCha20Poly1305 AEAD, as specified in RFC7539, with its nonce being composed of 32 bits of zeros followed by the 64-bit little-endian value of counter

- XAEAD(key, nonce, plain text, auth text): XChaCha20Poly1305 AEAD, with a random 24-byte nonce
- AEAD_LEN(plain len): plain len + 16
- HMAC(key, input): HMAC-Blake2s(key, input, 32), returning 32 bytes of output
- MAC(key, input): Keyed-Blake2s(key, input, 16), returning 16 bytes of output
- HASH(input): Blake2s(input, 32), returning 32 bytes of output
- TAI64N(): TAI64N timestamp of current time which is 12 bytes
- CONSTRUCTION: the UTF-8 value Noise_IKpsk2_25519_ChaChaPoly_BLAKE2s, 37 bytes
- IDENTIFIER: the UTF-8 value WireGuard v1 zx2c4 Jason@zx2c4.com, 34 bytes
- LABEL_MAC1: the UTF-8 value mac1----, 8 bytes
- LABEL_COOKIE: the UTF-8 value cookie--, 8 bytes

First Message: Initiator to Responder

The initiator sends this message:

```
msg = handshake_initiation {
    u8 message_type
    u8 reserved_zero[3]
    u32 sender_index
    u8 unencrypted_ephemeral[32]
    u8 encrypted_static[AEAD_LEN(32)]
    u8 encrypted_timestamp[AEAD_LEN(12)]
    u8 mac1[16]
    u8 mac2[16]
}
```

The fields are populated as follows:

```
initiator.chaining_key = HASH(CONSTRUCTION)
initiator.hash = HASH(HASH(initiator.chaining_key || IDENTIFIER) || responder.static_public)
initiator.ephemeral_private = DH_GENERATE()
msg.message_type = 1
msg.reserved_zero = { 0, 0, 0 }
msg.sender_index = little_endian(initiator.sender_index)

msg.unencrypted_ephemeral = DH_PUBKEY(initiator.ephemeral_private)
initiator.hash = HASH(initiator.hash || msg.unencrypted_ephemeral)

temp = HMAC(initiator.chaining_key, msg.unencrypted_ephemeral)
initiator.chaining_key = HMAC(temp, 0x1)

temp = HMAC(initiator.chaining_key, DH(initiator.ephemeral_private, responder.static_public))
initiator.chaining_key = HMAC(temp, 0x1)
key = HMAC(temp, initiator.chaining_key || 0x2)

msg.encrypted_static = AEAD(key, 0, initiator.static_public, initiator.hash)
initiator.hash = HASH(initiator.hash || msg.encrypted_static)

temp = HMAC(initiator.chaining_key, DH(initiator.static_private, responder.static_public))
initiator.chaining_key = HMAC(temp, 0x1)
key = HMAC(temp, initiator.chaining_key || 0x2)
```

```
msg.encrypted_timestamp = AEAD(key, 0, TAI64N(), initiator.hash)
initiator.hash = HASH(initiator.hash || msg.encrypted_timestamp)
```

```
msg.mac1 = MAC(HASH(LABEL_MAC1 || responder.static_public), msg[0:offsetof(msg.mac1)])
if (initiator.last_received_cookie is empty or expired)
    msg.mac2 = [zeros]
else
    msg.mac2 = MAC(initiator.last_received_cookie, msg[0:offsetof(msg.mac2)])
```

When the responder receives this message, he decrypts and does all the above operations in reverse, so that the state is identical.

Second Message: Responder to Initiator

The responder sends this message, after processing the first message above and applying the same operations to arrive at an identical state:

```
msg = handshake_response {
    u8 message_type
    u8 reserved_zero[3]
    u32 sender_index
    u32 receiver_index
    u8 unencrypted_ephemeral[32]
    u8 encrypted_nothing[AEAD_LEN(0)]
    u8 mac1[16]
    u8 mac2[16]
}
```

The fields are populated as follows:

```
responder.ephemeral_private = DH_GENERATE()
msg.message_type = 2
msg.reserved_zero = { 0, 0, 0 }
msg.sender_index = little_endian(responder.sender_index)
msg.receiver_index = little_endian(initiator.sender_index)
```

```
msg.unencrypted_ephemeral = DH_PUBKEY(responder.ephemeral_private)
responder.hash = HASH(responder.hash || msg.unencrypted_ephemeral)
```

```
temp = HMAC(responder.chaining_key, msg.unencrypted_ephemeral)
responder.chaining_key = HMAC(temp, 0x1)
```

```
temp = HMAC(responder.chaining_key, DH(responder.ephemeral_private, initiator.ephemeral_
responder.chaining_key = HMAC(temp, 0x1)
```

```
temp = HMAC(responder.chaining_key, DH(responder.ephemeral_private, initiator.static_publi
responder.chaining_key = HMAC(temp, 0x1)
```

```
temp = HMAC(responder.chaining_key, preshared_key)
responder.chaining_key = HMAC(temp, 0x1)
temp2 = HMAC(temp, responder.chaining_key || 0x2)
key = HMAC(temp, temp2 || 0x3)
responder.hash = HASH(responder.hash || temp2)
```

```
msg.encrypted_nothing = AEAD(key, 0, [empty], responder.hash)
responder.hash = HASH(responder.hash || msg.encrypted_nothing)
```

```
msg.mac1 = MAC(HASH(LABEL_MAC1 || initiator.static_public), msg[0:offsetof(msg.mac1)])
if (responder.last_received_cookie is empty or expired)
    msg.mac2 = [zeros]
else
    msg.mac2 = MAC(responder.last_received_cookie, msg[0:offsetof(msg.mac2)])
```

When the initiator receives this message, he decrypts and does all the above operations in reverse, so that the state is identical.

Data Keys Derivation

After the above two messages have been exchanged, keys are calculated by the initiator and responder for sending and receiving data:

```
temp1 = HMAC(initiator.chaining_key, [empty])
temp2 = HMAC(temp1, 0x1)
temp3 = HMAC(temp1, temp2 || 0x2)
initiator.sending_key = temp2
initiator.receiving_key = temp3
initiator.sending_key_counter = 0
initiator.receiving_key_counter = 0
```

```
temp1 = HMAC(responder.chaining_key, [empty])
temp2 = HMAC(temp1, 0x1)
temp3 = HMAC(temp1, temp2 || 0x2)
responder.receiving_key = temp2
responder.sending_key = temp3
responder.receiving_key_counter = 0
responder.sending_key_counter = 0
```

And then all previous chaining keys, ephemeral keys, and hashes are zeroed out.

Subsequent Messages: Exchange of Data Packets

The initiator and the responder exchange this packet for sharing encapsulated packet data:

```
msg = packet_data {
    u8 message_type
    u8 reserved_zero[3]
    u32 receiver_index
    u64 counter
    u8 encrypted_encapsulated_packet[]
}
```

The fields are populated as follows:

```
msg.message_type = 4
msg.reserved_zero = { 0, 0, 0 }
msg.receiver_index = little_endian(responder.sender_index)
```

```
encapsulated_packet = encapsulated_packet || zero padding in order to make the length a multiple of 16
counter = initiator.sending_key_counter++
msg.counter = little_endian(counter)
msg.encrypted_encapsulated_packet = AEAD(initiator.sending_key, counter, encapsulated_packet)
```

The responder uses his responder.receiving_key to read the message.

DoS Mitigation

We require authentication in the first handshake message sent because it does not require allocating any state on the server for potentially unauthentic messages. In fact, the server does not even *respond at all* to an unauthorized client; it is silent and invisible. The handshake avoids a denial of service vulnerability created by allowing any state to be created in response to packets that have not yet been authenticated.

This, however, introduces the issue of having authentication in the first packet: it is always open to a replay attack. An attacker could replay initial handshake messages to trick the server into regenerating its ephemeral key, thereby disconnecting the legitimate client connection (though not affecting the security of any messages). For that reason, we include a [TAI64N](#) timestamp in the first message. The server keeps track of the greatest timestamp received *per client* and discards packets containing timestamps less than or equal to it. If the server restarts and loses this state, that is not a problem: an initial packet from earlier can be replayed, but it could not possibly disrupt any ongoing sessions, since the server has just restarted. Once clients reconnect to the server after its restart, they will be using greater timestamps, invalidating the previous ones. This timestamp ensures that an attacker can't disrupt a current session between client and server.

Furthermore, computing the DH() function is CPU intensive. In order to fend off a CPU-exhaustion attack, if the server is under load, it may choose to not process handshake messages, but instead respond with a cookie reply packet. In order for the server to remain silent unless it receives a valid packet, while under load, all messages are required to have a MAC that combines the receiver's public key and optionally the PSK as the MAC key. When the server is under load, it will only accept packets that additionally have a second MAC of the prior bytes of the message that utilize the cookie as the MAC key. We therefore compute msg.mac1 and msg.mac2 as seen in the handshake messages above. Cookies expire after two minutes and are a MAC of the sender's IP address using a changing (every two minutes) server secret as the MAC key. This allows for proof of IP ownership, which can then be rate limited properly. The server, after computing these MACs as well and comparing them to the ones received in the message, must reject messages with an invalid msg.mac1 and when under load must reject messages with an invalid msg.mac2.

Cookie Reply Packet

As mentioned above, when a message with a valid msg.mac1 is received, but msg.mac2 is all zeros or invalid and the server is under load, the server may send a cookie reply packet as follows:

```
msg = packet_cookie_reply {
    u8 message_type
    u8 reserved_zero[3]
    u32 receiver_index
    u8 nonce[24]
    u8 encrypted_cookie[AEAD_LEN(16)]
}
```

```
}
```

```
msg.message_type = 3  
msg.reserved_zero = { 0, 0, 0 }  
msg.receiver_index = little_endian(initiator.sender_index)  
msg.nonce = RAND(24)  
cookie = MAC(responder.changing_secret_every_two_minutes, initiator.ip_address)  
msg.encrypted_cookie = XAEAD(HASH(LABEL_COOKIE || responder.static_public), msg.nonce,
```

Nonce Reuse & Replay Attacks

Nonces are never reused. A 64bit counter is used, and cannot be wound backward. UDP, however, sometimes delivers messages out of order. For that reason we use a sliding window, in which we keep track of the *greatest* counter received and a window of roughly 2000 prior values, checked after verifying the authentication tag. This avoids replay attacks while ensuring nonces are never reused and that UDP can maintain out-of-order delivery performance.

DiffServ Considerations

The "DiffServ" bits in an IP packet are generally split into two portions: one describing the quality of service, via the DSCP value, and the other containing bits used for Explicit Congestion Notification (ECN). All handshake packets have a DSCP value of 0x88 (AF41), so that these packets are the least likely to be dropped, as they're essential for the control functionality of the tunnel, and the ECN is set to 00. All transport data packets have a DSCP value of 0, because the DSCP value of the inner packet is *never* copied to the outer packet, so that we don't leak information about the data inside the encrypted inner packet. However, we do copy the ECN bits to and from the inner packets, in accordance with the logic described in RFC6040.